

CASE STUDY 01

Python Curriculum Unit

Designing a structured, ADDIE-based coding curriculum for global learners aged 11–16

Athira A | Instructional Designer Portfolio | github.com/Athiraajith91

OVERVIEW

This case study documents the design and development of a complete Python curriculum unit — independently created as a portfolio project to demonstrate instructional design capability. The curriculum was built using the ADDIE framework, aligned to Bloom's Taxonomy, and designed for delivery in a 1:1 online EdTech environment with international learners.

Project Type	Self-directed curriculum development portfolio project
Target Audience	Beginner Python learners, Grades 6–10, aged 11–16
Delivery Mode	Online 1:1 and small group sessions
Duration	5-lesson unit — approximately 5 hours of instruction
Tools Used	ADDIE Framework, Bloom's Taxonomy, Google Docs, PDF

The Problem — What Gap Did This Curriculum Address?

During 5+ years of teaching to international learners, I consistently noticed the same problem — most available Python resources for beginners either moved too fast (assuming prior programming knowledge) or were too shallow (covering syntax without teaching logical thinking). Young learners aged 11–16 needed a curriculum that:

- Started with the WHY before the HOW — building genuine curiosity before syntax
- Used real-world analogies to make abstract concepts like loops and conditionals concrete
- Included guided practice at every stage — not just watching, but doing

- Had clear learning objectives so both teachers and students knew what success looked like
- Progressively built complexity — each lesson unlocking the next

“Most Python resources teach syntax. This curriculum teaches thinking. The syntax comes naturally when the thinking is right”.

Phase 1 — Analysis (ADDIE)

Before designing a single lesson, I conducted an informal needs analysis based on my teaching experience:

- Learner profile: Beginners with zero programming experience, aged 11–16, motivated but easily discouraged by complexity
- Learning gap: Students could copy code, but could not write or debug independently
- Environmental factors: Online delivery, 45–60 minute sessions, no in-person support
- Prerequisite knowledge: Basic computer literacy only — no prior coding assumed
- Desired outcomes: Learners who can independently write, read, and debug basic Python programs

Phase 2 — Design

Using Bloom's Taxonomy as the framework, I designed learning objectives at six cognitive levels across the 5-lesson unit:

Remember	Define Python, IDE, variable, data type, conditionals, operators and input
Understand	Explain what each concept does using a real-world analogy
Apply	Write code using each concept in a guided exercise
Analyse	Identify errors in the sample code and explain why they occur
Evaluate	Compare two approaches to solving the same problem
Create	Design and build an original mini project using all concepts

The lesson sequence was designed using backward design — starting with the final project (what learners would CREATE) and working backward to the foundational knowledge needed:

- Lesson 1: Introduction to Python — what it is, why it matters, first program
- Lesson 2: Variables and Data Types — storing and using information
- Lesson 3: Input and Output — making programs interactive
- Lesson 4: Conditional Statements — teaching the computer to make decisions
- Lesson 5: Mini Project — learner-chosen capstone applying all concepts

Phase 3 — Development

Each lesson was developed with the following components:

- Lesson plan with timing guide, materials list, and teacher notes
- Hook activity — a question, puzzle, or real-world scenario to activate curiosity
- Concept explanation with at least one non-coding analogy (e.g., conditionals explained using marks and age)
- Worked example — teacher models the code with narration
- Guided practice — student completes similar task with scaffolding
- Independent challenge — student applies concept without scaffolding
- Exit ticket — one question checking core understanding before the session ends

Key Instructional Design Decisions

Every design decision was driven by one question: What does this learner need to feel capable, not just compliant?

- Analogies before syntax: Every concept was explained through a relatable real-world comparison before any code was shown — reducing cognitive load for beginners

- Worked examples: Research on cognitive load theory shows that beginners learn faster from seeing worked examples than from problem-solving alone. Every lesson modelled the task before asking students to try
- Choice-based final project: Giving learners agency over their capstone project significantly increases intrinsic motivation — a finding supported by Self-Determination Theory
- Spiral curriculum: Core concepts like variables and conditionals were revisited in later lessons, deepening understanding through application rather than repetition

Outcomes and Reflection

KEY OUTCOMES

- Complete 5-lesson curriculum unit with lesson plans, activities, and assessments
- Learning objectives mapped to all 6 levels of Bloom's Taxonomy
- Consistent lesson structure enabling delivery by any instructor — not just the designer
- Portfolio artifact demonstrating end-to-end ADDIE curriculum design process
- Framework transferable to other coding languages — JavaScript, Scratch, HTML

If I were to develop this further, I would add a formal pre-assessment to diagnose learner starting points and differentiated lesson tracks for fast and slow progressors. I would also design a student-facing progress tracker aligned to the learning objectives.

What This Case Study Demonstrates

- Ability to apply ADDIE from analysis through development — not just content creation
- Understanding of Bloom's Taxonomy applied practically to lesson sequencing
- Backward design thinking — starting with the end goal and working backward
- Learner-centred design — every decision justified by learner needs, not convenience